STUDENT CERTIFICATE

Name: **Sem: V**

Register No:

1	0	9	C	S					

Institution: 109 - Government Polytechnic Kalaburagi.

This is to certify that above mention student has successfully carried out lab practice sessions in the **Full Stack Development—20CS52I** integrated course during the **Academic Year 2026-27.**

Course Cohort **Program Coordinator**
Farheen Jahan Aara

Examiners Signature

1. **_____**
2. **_____**

List of Experiments

Sl. No.	Name of the Experiment	Date of Experiment	Remarks
1	**Write and identify business entity integration of a user story and test cases (Hotel booking, Flipkart).**		
2	**design and implement authentication workflow using openID**		
3	**Create a Repository named mini project-1 and Push the same to Github**		
4	**Creating a cloud account.**		
5	**Creating Web Application.**		
6	**Java script for form validation.**		
7	**create a simple Full Stack application using** React**,** Node.js**, and** Express **to register student details**		
8	**Create and Build Pipeline for a Simple Web Application (Word Count)**		
9	**Create and run first program in Typescript.**		
10	**Build an application with spring boot**		
11	**How to create RESTful services and install postman test create APIs with the help of postman .**		
12	**Write the program to Create React Router in react.**		
13	**Build User Authentication and Authorization using Spring Security**		
14	**Spring Boot application with a REST controller that add two numbers.(in Spring tool suite)**		
15	**Create Student.java – an entity class with student information. Design a Rest Controller with GET, PUT, POST and DELETE Restful APIs.**		
16	**Write a java program to connect to a mysql database, retrieve data from a table and display it in the console.**		
17	**Create a Java application to insert new records into an existing.**		
18	**Write a JDBC program that uses a prepared statement to update.**		
19	**Perform the following operations using MongoDB**		
20	**Perform the following operations using MongoDB Create Library management database.**		
21	**10 Hours Training certificate(InfosysSpringboard)**		

1.**Write and identify business entity integration of a user story and test cases (Hotel booking, Flipkart).**

1.**Hotel Booking** is an online Hotel room booking application that helps the users to book a room for staying at a particular place across Karnataka. This application allows users to login for booking a room. Users can search for the room at a hotel for a specific location. Once found, users can check the availability of a room for specific dates. Users can book a hotel required date. Once booked, users can get the booking details. Identify and write the user stories for this application.Write test cases for the above application.

Steps for Any Project

1. **Identify the user**
   - Customer
   - Admin
   - Student
   - Seller

2. **Identify the feature**
   - Login
   - Registration
   - Search
   - Booking
   - Payment
   - Cart
   - Profile Update

3. **Write the user story**
   - As a...
   - I want to...
   - So that...

4. **Write acceptance criteria**
   - Define the conditions that must be met for the feature to be complete.

5. **Write test cases**
   - Test Case ID
   - Test Scenario
   - Test Steps
   - Test Data (if applicable)
   - Expected Result
   - Status (Pass/Fail)

Write test cases for the above application.

1. Users are able to access the Hotel Booking Home page.
2. Validate the hotel booking Home page is rendered correctly for desktop as per the design specifications
. 3. Validate the hotel booking Home page is rendered correctly for tablet as per the design specifications.
4. Validate the hotel booking Home page is rendered correctly for a mobile device as per the design specifications
. 5. Validate hotel search fields are visible on screen.
6. User searches for a holiday to any place across Karnataka for a family of 2 adults and 2 children and makes a payment (End to End Test).
7. User makes a successful payment for their hotel booking.
8. User makes an unsuccessful payment of their hotel booking.
9. Hotel Room Unavailability – User searches for dates that are unavailable and system recommends 13 alternative dates or room types
10. Users want to Amend an existing booking by adding an additional feature (e.g. increase length of stay / adding breakfast).
11. User wants to Verify the itinerary and print a Paper version
12. User cancels their booking and system refunds money – Test Refund Conditions
13. User cancels their booking and the system does NOT refund money – Test Refund Conditions
14. User wants to make a group booking
15. User wants to Validate Booking Page displays correct booking data– Visual check
16. Confirm Payment Page is displayed when the user selects “Make Payment”.
17. End to End Test of Hotel Booking Engine.

a) Flipkart is an online shopping application that helps its users to buy variety of authentic products. This application allows users to log in for buying products. Users cansearch for a product, sort the product list based on rating or price. Users can select the items and add them to the cart. Once the selection is done, users can go to the cart page for payment. Identify and write the user stories for this application.

Registration

1.Sign-up:

As a shopper, I want to sign up for the Flipkart application through a New user form, so that I Can get access to buy a variety of products.

Acceptance Criteria:

1. While signing up-Valid Phone Number/Email Id and OTP/Password.

2. If sign up is successful, it will get automatically logged in. 3. If I am trying to sign up with an invalid phone number/Email Id, I will receive an error message to enter valid information. 4. If we are trying to sign up with an existing phone number/Email Id, we will receive an error message saying "you are already registered."

2. Login

As an authorized shopper, I want to log in for the Flipkart application, so that I can have access to the application for searching and buying products.

Acceptance Criteria:

1. While logging in, Phone number/Email Id and OTP/Password are required.
2. After successful login, it will be redirected to the main page.
3. If we are trying to login with an incorrect mobile number/Email Id or OTP/Password, then an error message will be displayed as "invalid credentials".

3.View a List of Products

As a Shopper I want to view a list of products so I can select some items to purchase

Acceptance Criteria:

1. See a thumbnail image for each product
2. Click to view details for product
3. Add to cart from detail page
4. Search for a product
5. View products by category

4.Review a Cart

As a Shopper I want to review my cart so I can make adjustments prior to checkout Acceptance Criteria:

1. View quantities and items in the cart
2. See a total cost before tax and shipping
3. Remove items
4. Adjust quantity of items
5. Click to navigate to a product detail page

5.Check out

As a Shopper I want to check out so I can get my products shipped to me.

Acceptance Criteria:

1. Trigger checkout from any page, if there are items in the cart
2. Enter a shipping address
3. Enter a billing address
4. Enter a credit card number
5. Show total including tax and shipping before finalizing.
6. Show Confirmation message after finalizing

7. Verify payment through our payment processor

6. Review Orders

As a Shopper I want to review my orders so I can see what I have purchased in the past.

Acceptance Criteria:

1. View a list of open and completed orders
2. See the status of the order
3. Navigate to the details of the order
4. Include a tracking number if the order is shipped but not delivered
5. Contact customer service about an order from the details page

7. Logout

As a Shopper, I want to log out of the Flipkart application, so that I can prevent unauthorised access of my profile.

Acceptance Criteria: When I log out of my account, I will be redirected to the log-in page.

Write test cases for the above application.

Test Cases for Flipkart Website Login Page

Check that there are proper validations on the Login Page.

Check for an error message if the Email, password, or any required field is left blank. Check the validations on the Email and password.

When you log in, check that you stay logged in as you browse products. Also, you need to test the behaviour.

When the user doesn't interact with the site for some time, then the user has been logged out after the session times out.

When you are logged in, log out and make sure you are logged out and that you cannot access any of the account's pages.

Test Cases for Flipkart Website Search Page

Check that the products displayed are related to what was searched for.

Check that the products should display an image, name, price, and maybe customer ratings and number of reviews.

the more relevant product for the search term is displayed on the top for a particular search 15 term.

Check that all items on the next page are different from the previous page, i.e., no duplicates.

Check that when both sorts and filters have been applied, they remain as we paginate or more products are loaded.

Check that the count of products is correctly displayed on the search result page for a particular search term.

Check that filtering functionality correctly filters products based on the filter applied.

Check that filtering works correctly on category pages.

Check that filtering works correctly on the search result page.
Check that the correct count of total products is displayed after a filter is applied. Check that all the sort options work correctly – correctly sort the products based on the sort option chosen.
Check that sorting works correctly on the category pages.
Check that sorting works correctly on the search result page.
Check that sorting works correctly on the pages containing the filtered result, after applying filters.
Check that the product count remains intact irrespective of the sorting option applied.

Test Cases for Flipkart Website Shopping Cart

Users should be able to add a product to the cart.
Item count should be incremented when the user adds the same product again.
Taxes should be applied according to the delivery location.
Users should be able to add items to the cart.
Users should be able to update items in the cart.
Checkout should happen successfully for the items added to the cart.
Shipping costs for different products are added to the cart.
Coupons should be applied successfully to the cart.
The cart should retain the items even when the app is closed.

Payment Page Test Cases

Check that After filling the shipping address and payment, the product is purchased successfully.
Check that Different payment types should be present, e.g., Credit Card, PayPal, Bank Transfers, Instalments, etc. Check the security of the client's card details when entered for payment.

Test Case for Flipkart Website– Post Order Page

Email and order id should be sent after placement of order. Users should be able to cancel the order.
There should be a facility for users to track the order. Users should be able to return/replace the product post-delivery.

2.Design and implement authentication workflow using open ID

Step 1: Create Project

```
mkdir openid-demo
cd openid-demo
npm init -y
```

Step 2: Install Package

```
npm install express openid-client
```

Step 3: Create server.js

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
 res.send("<h2>OpenID Login Successful!</h2>");
});
app.listen(3000, () => {
  console.log("Server running on http://localhost:3000");
});
```

Step 4: Run the Project

```
node server.js
```

Open your browser and visit:

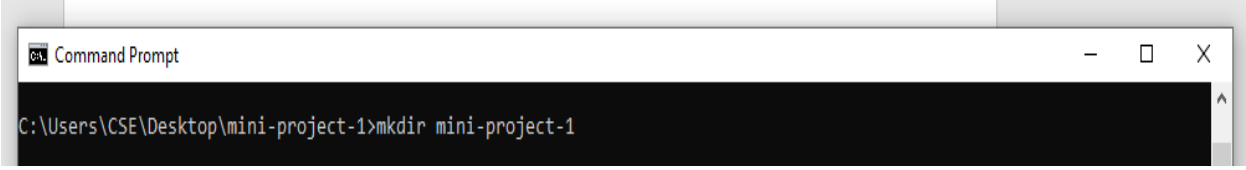
```
http://localhost:3000
```

Output:

OpenID Login Successful!

3. Create a Repository named mini project-1 and Push the same toGitHub

`mkdir mini-project-1`



`cd mini-project-1`



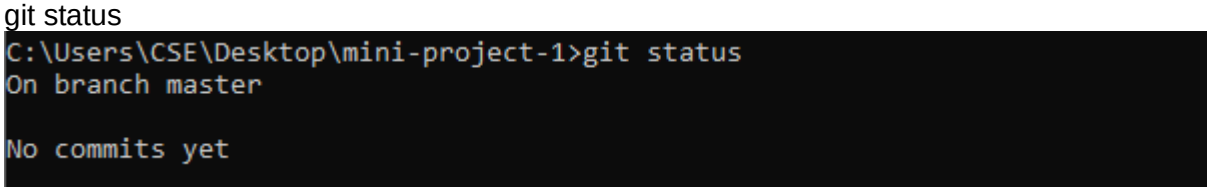
`git init` → Creates a new Git repository.

git `init`



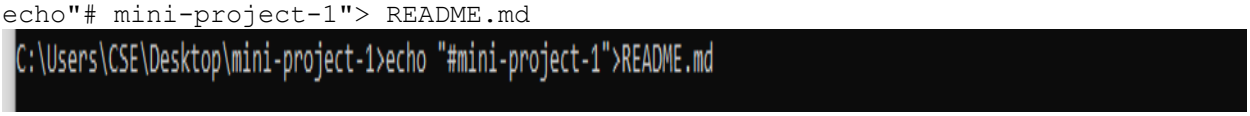
`git status` → Shows modified and staged files.

git status



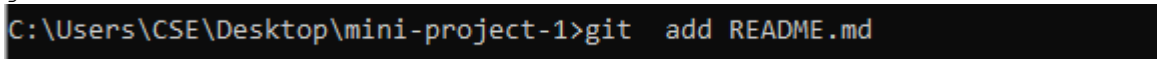
`Git echo` → To Create a File

`echo"# mini-project-1"> README.md`



`git add .` → Stages all changes

`git add`



`git commit -m "message"` → Saves changes to the repository.

git `commit` -m"Initial commit"

```
C:\Users\CSE\Desktop\mini-project-1>git commit -m "Initial Commit"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'CSE@DESKTOP-TLLN9E9.(none)')
```

git `branch` -M `main`

```
C:\Users\CSE\Desktop\mini-project-1>git branch -M main

C:\Users\CSE\Desktop\mini-project-1>
```

`git clone <URL>` → Copies an existing repository

git `remote add origin` https://github.com/your-username/mini-project-1.git

```
C:\Users\CSE\Desktop\mini-project-1>git remote add origin https://github.com/Farheen39-max/mini-project-1

C:\Users\CSE\Desktop\mini-project-1>git remote -v
origin  https://github.com/Farheen39-max/mini-project-1 (fetch)
origin  https://github.com/Farheen39-max/mini-project-1 (push)
```

`git push origin main → Uploads commits to the remote repository`

git `push` -u `origin main`

```
C:\Users\CSE\Desktop\mini-project-1>git push -u origin main
error: src refspec main does not match any
error: failed to push some refs to 'https://github.com/Farheen39-max/mini-project-1'

C:\Users\CSE\Desktop\mini-project-1>
```

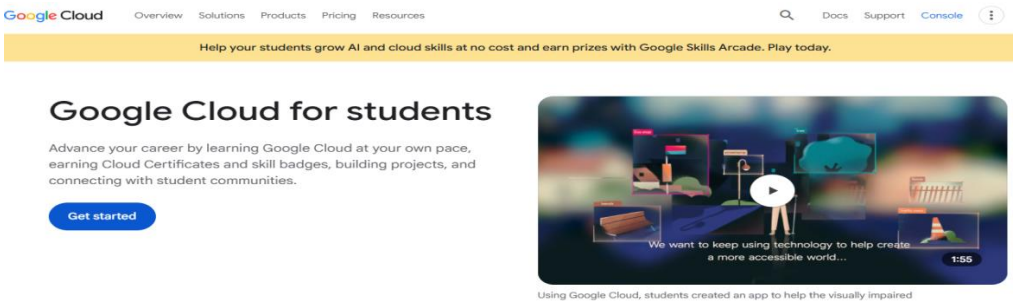
`git log` → Displays the complete commit history.
`git log --oneline` → Displays a concise commit history.

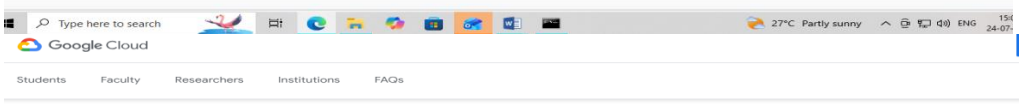
git log

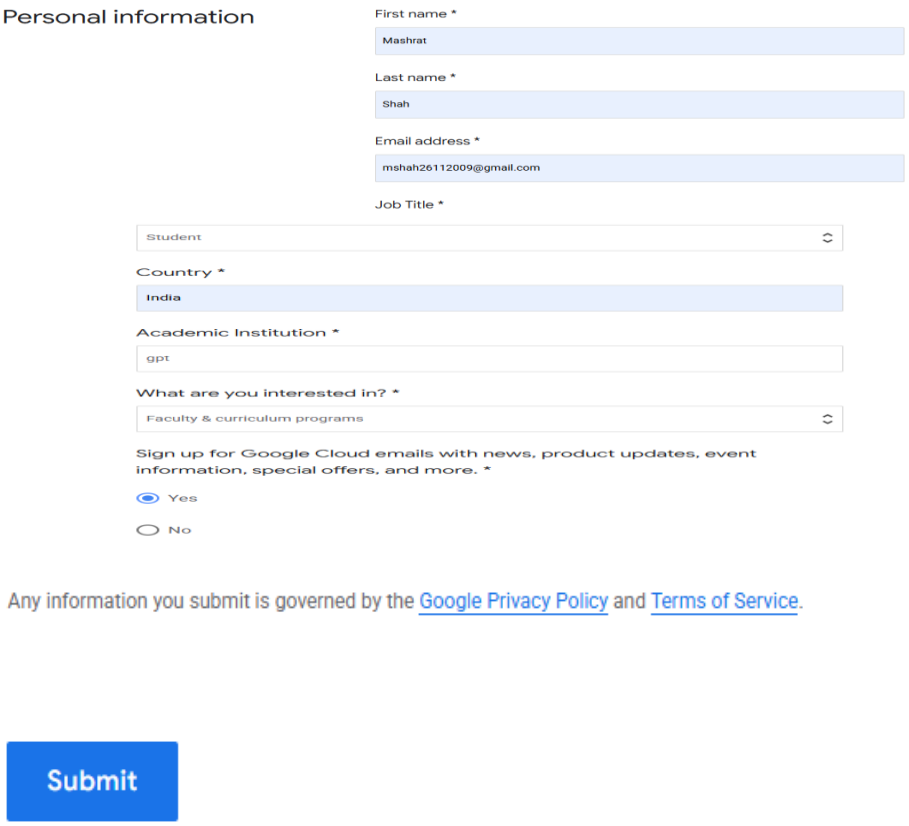
```
C:\Users\CSE\Desktop\mini-project-1>git log
fatal: your current branch 'main' does not have any commits yet

C:\Users\CSE\Desktop\mini-project-1>
```

4.Creating a cloud account.







5.CreatingWeb Application.

```
<!DOCTYPE html>
<html>
<head>
<title>Student Portal</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<h1>Welcome to Student Portal</h1>

<p>Name: Summit</p>
<p>Course: Full Stack Development</p>
<p>College: GPT KALABURAGI</p>

<button onclick="showMessage()">Click Me</button>

<script src="script.js"></script>

</body>
</html>
```

CSS

```
body{
    font-family: Arial;
    background:#f4f4f4;
    text-align:center;
}

h1{
    color:blue;
}

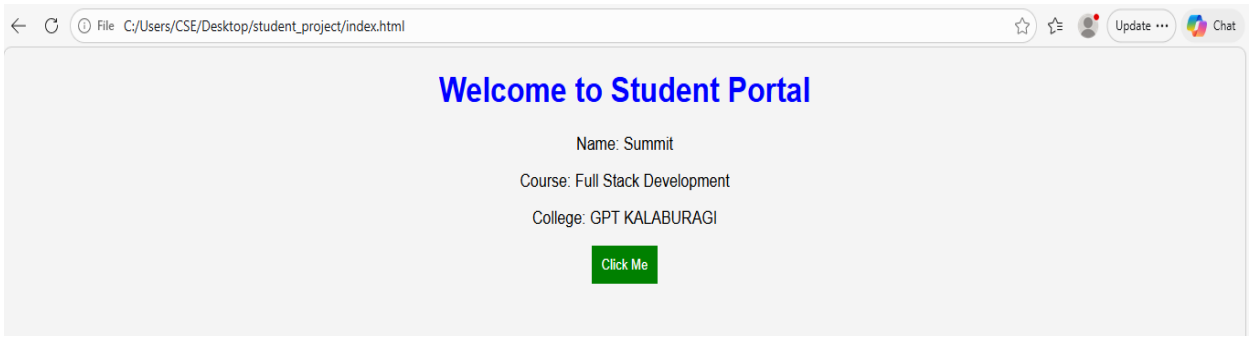
button{
    background:green;
    color:white;
    padding:10px;
    border:none;
    cursor:pointer;
}

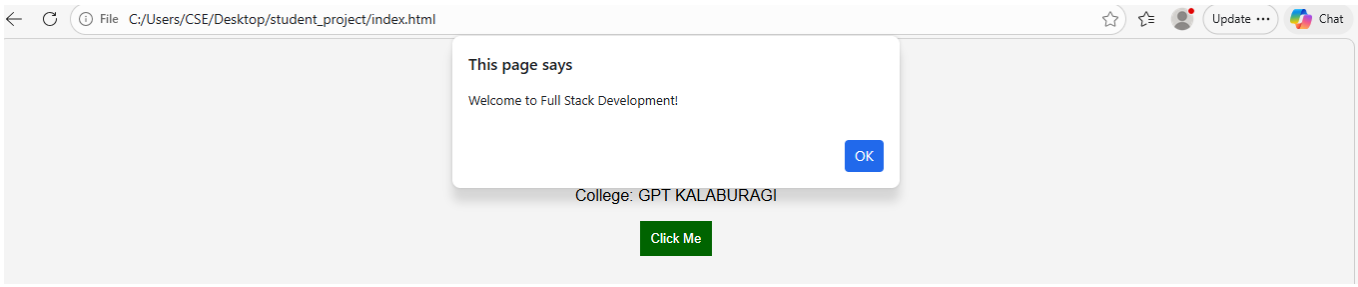
button:hover{
    background:darkgreen;
}
```

JavaScript

```
function showMessage(){
   alert("Welcome to Full Stack Development!");
}
```

OUTPUT:





6.Write a Java script for form validation.

```
<!DOCTYPE html>
<html>
<head>
<title>Form Validation with Result</title>
<style>
      body{
         font-family: Arial;
         text-align: center;
         background-color: lightcyan;
      }
      form{
         width:300px;
         margin:auto;
         padding:20px;
         background:pink;
         border:2px solid black;
      }
      input{
         width:90%;
         padding:8px;
         margin:8px;
      }
      #result{
         margin-top:20px;
         font-size:18px;
         color:blue;
      }
</style>
</head>
<body>

<h2>Student Form</h2>

<form id="studentForm">
<input type="text" id="name" placeholder="Enter Name"><br>
<input type="text" id="usn" placeholder="Enter USN"><br>
<input type="text" id="branch" placeholder="Enter Branch"><br>
<input type="submit" value="Submit">
</form>

<div id="result"></div>

<script>
document.getElementById("studentForm").onsubmit = function(event) {
   event.preventDefault();
```

```
    let name = document.getElementById("name").value;



 let usn = document.getElementById("usn").value;
    let branch = document.getElementById("branch").value;

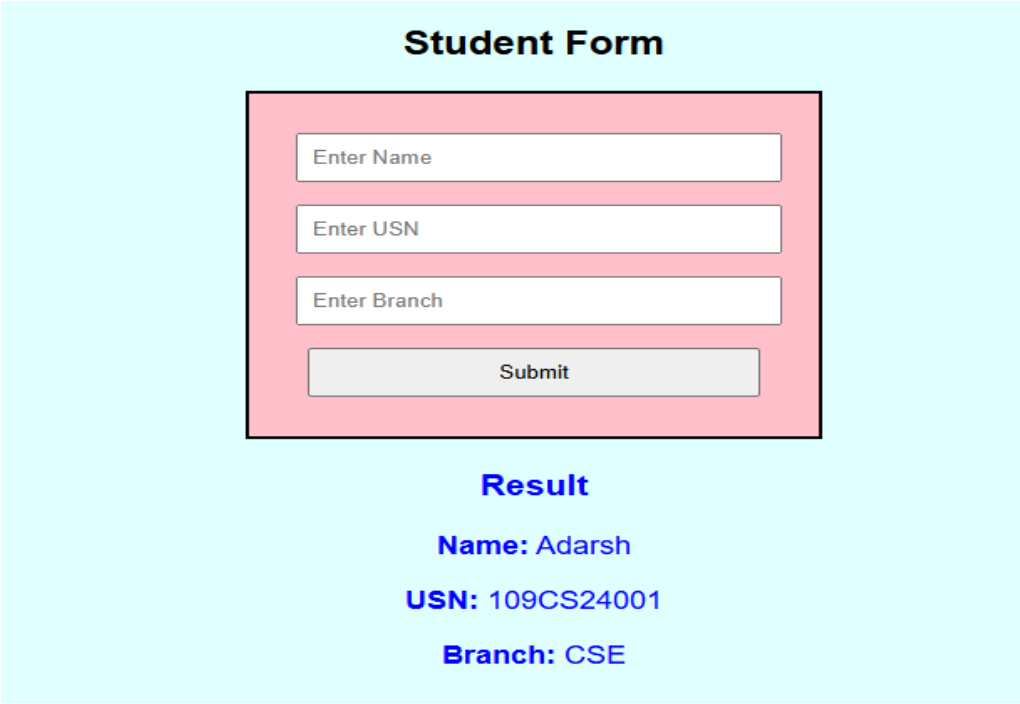
    if (name == "" || usn == "" || branch == "") {
        alert("Please fill all the fields.");
        return;
    }

    document.getElementById("result").innerHTML =
        "<h3>Result</h3>" +
        "<p><b>Name:</b> " + name + "</p>" +
        "<p><b>USN:</b> " + usn + "</p>" +
        "<p><b>Branch:</b> " + branch + "</p>";

    document.getElementById("studentForm").reset();
}
</script>

</body>
</html>
```

OUTPUT:



7.create a simple Full Stack application using React, Node.js, and Express to register student details.

Project Structure:

```
student-app/
├── backend/
│   └── server.js
└── frontend/
    └── src/
        └── App.js
```

Step 1: Backend save filename:sever.js

```
npm install express cors
const express = require("express");
const cors = require("cors");

const app = express();

app.use(cors());
app.use(express.json());

app.post("/student", (req, res) => {
  res.json({
    message: "Student Registered Successfully"
    student: req.body
  });
});

app.listen(5000, () => {
  console.log("Server Running on Port 5000");
});
```

Run the backend:

node server.js

Step 2: Frontend (React)

Install Axios:
npm install axios

save filename app.js

```
import { useState } from "react";
import axios from "axios";
```

```
function App() {
  const [name, setName] = useState("");

  const submit = async () => {
    const res = await axios.post("http://localhost:5000/student", {
      name: name
    });
    alert(res.data.message);
  };

  return (
<div style={{ textAlign: "center", marginTop: "50px" }}>
<h2>Student Registration</h2>

<input
        type="text"
        placeholder="Enter Name"
        onChange={(e) => setName(e.target.value)}
      />

<br /><br />

<button onClick={submit}>Submit</button>
</div>
  );
}

export default App;
```

Run the frontend:

npm start

OUTPUT:

Student Registration

Enter Name: Abhay

[Submit]

Message:
Student Registered Successfully

8.Create and Build Pipeline and react for a Simple Web Application (Word Count)

Step 1: Create React Project

npx create-react-app wordcount-app
cd wordcount-app

Step 2: Replace App.js

```
import { useState } from "react";

function App() {
  const [text, setText] = useState("");

  return (
<div style={{ textAlign: "center", marginTop: "50px" }}>
<h2>Word Count App</h2>

<textarea
       rows="5"
       cols="30"
       placeholder="Type here..."
       onChange={(e) => setText(e.target.value)}
></textarea>

<h3>Words: {text.trim() === "" ? 0 : text.trim().split(/\s+/).length}</h3>
</div>
  );
}
export default App;
```

Step 3: Run the Application

npm start

Open the browser:

http://localhost:3000

OUTPUT:

```
-------------------------
     Word Count App

Type here:
Hello React Students

Words: 3
-------------------------
```

9. create and run first program in Typescript.

Step 1: Install TypeScript

npm install -g typescript

Step 2: Create a File

Create a file named **`hello.ts`**.

Step 3: Write the Program

let message: string = "Hello, World!";
console.log(message);

Step 4: Compile the Program

tsc hello.ts

Step 5: Run the Program

node hello.js

OUTPUT:

Hello, World!

10.Build an application with spring boot in vs code

Step 1: Create a Spring Boot Project

1.open vs code
2.press ctrl+shift+p
3. Search Spring Initializr: Create a Maven Project.
4.Select:
- .Language: Java
- .Spring Boot: latest stable version
- .Group: com.example
- .Artifact: HelloSpringBoot
- .Packaging: Jar
- .Java: 17 or newer

5.Select dependency:
- Spring Web

6.Choose a folder and open the generated project in VS Code.

7. Create the controller
Inside:
src/main/java/com/example/demo/

create a file named:
HelloController.java

Step 2: Create the Main Class

Save filename:hellocountroller.java

```
package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    @GetMapping("/")
    public String hello() {
        return "Hello, Spring Boot!";
    }
}
```

Step 3: Run the Application

Open the vs code terminal and type:

./mvnw spring-boot:run

Step 4: Open the Browser

Visit:

http://localhost:8080

OUTPUT:

Hello, Spring Boot!

11. How to create RESTful services and install postman test create APIs with the help of postman.

Step:1

1.open vs code
2.press ctrl+shift+p
3. Search Spring Initializr: Create a Maven Project.
4.Select:
.Language: Java
.Spring Boot: latest stable version
.Group: com.example
.Artifact: HelloSpringBoot
.Packaging: Jar
.Java: 17 or newer

5.Select dependency:
Spring Web
6.Choose a folder and open the generated project in VS Code.

7. Create the controller
Inside:
src/main/java/com/example/demo/

create a file named:
HelloController.java

Step 2: Create REST Controller

Create a file named **HelloController.java**

```
package com.example.demo;

import org.springframework.web.bind.annotation.*;

@RestController
public class HelloController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello, REST API!";
    }

    @PostMapping("/student")
```

```
    public String addStudent() {
        return "Student Added Successfully!";
    }
}
```

Step 3: Run the Application

Open the vs code terminal and type:

```
./mvnw spring-boot:run
```

Step 4: Install Postman

1. Download and install **Postman**.
2. Open the Postman application.

Step 5: Test the APIs

1. GET API

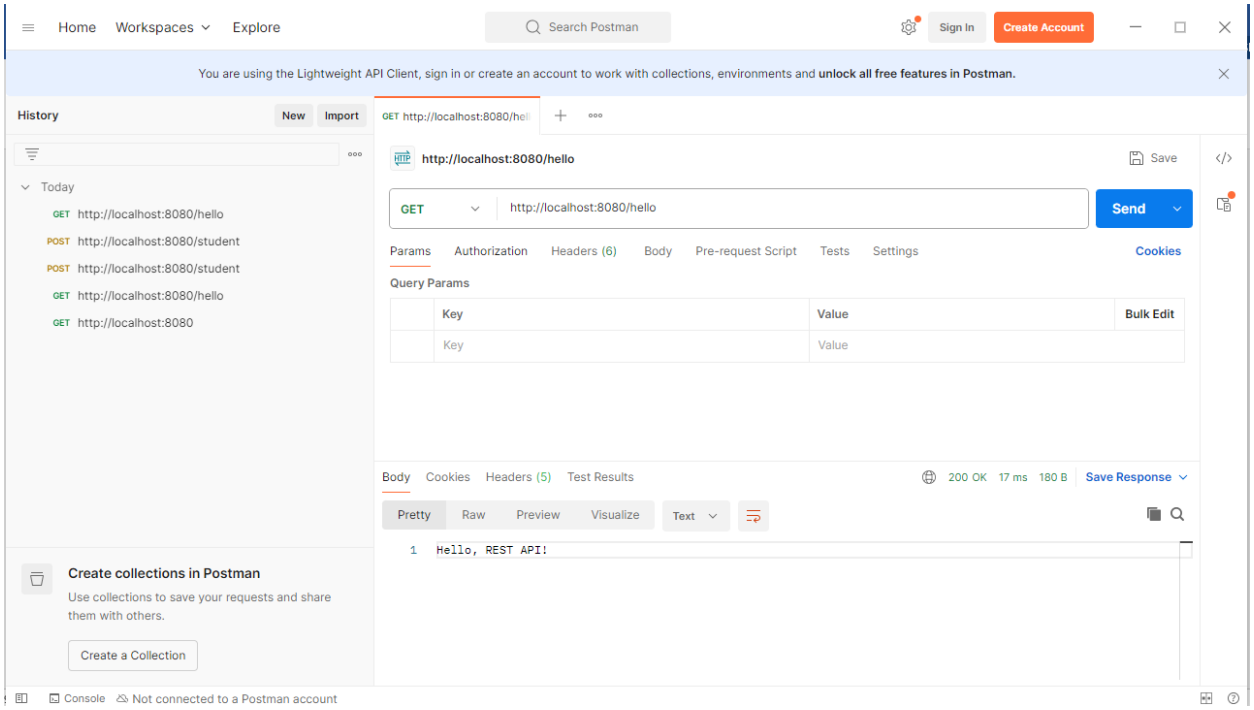
- Method: GET
- URL:

http://localhost:8080/hello

Click Send

Output:

Hello, REST API!



2. POST API

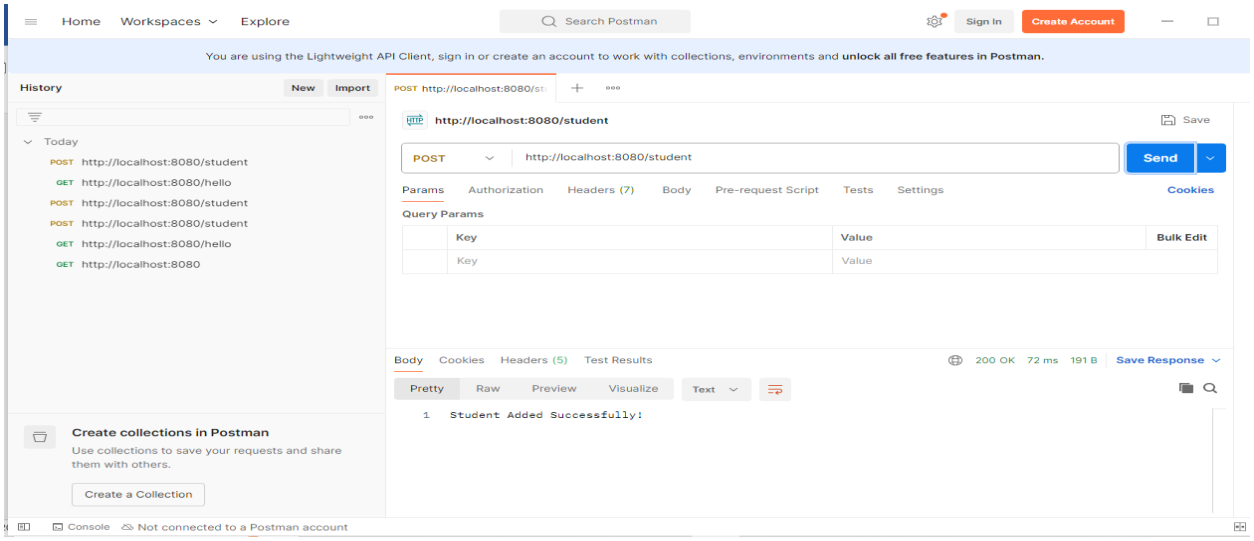
- Method: **POST**
- URL:

http://localhost:8080/student

Click **Send**

Output:

Student Added Successfully!


Home
Workspaces
Explore
Search Postman
Sign In
Create Account
You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.
History
New
Import
Today
POST http://localhost:8080/student
GET http://localhost:8080/hello
POST http://localhost:8080/student
POST http://localhost:8080/student
GET http://localhost:8080/hello
GET http://localhost:8080
Create collections in Postman
Use collections to save your requests and share them with others.
Create a Collection
http://localhost:8080/student
Save
POST
http://localhost:8080/student
Send
Params
Authorization
Headers (7)
Body
Pre-request Script
Tests
Settings
Cookies
Query Params
Key
Value
Bulk Edit
Body
Cookies
Headers (5)
Test Results
200 OK
72 ms
191 B
Save Response
Pretty
Raw
Preview
Visualize
Text
1 Student Added Successfully!
Console
Not connected to a Postman account

12. Write the program to Create React Router in react.

Step 1- Open Command prompt
Step 2 - create a folder
Step 3 - npx create-react-app react-router-demo
- cd react-router-demo
Step 4 - Install react router
- npx install react-router-demo
Step 5 - Open the project in VS code
Step 6 - create the page folder in vs code inside src(react-router-demo ->src ->pages)

2. Create a folder pages/
Inside src/, create a folder named pages and add these files:
Home.js
Blogs.js
Contact.js
NoPage.js
Layout.js

3. index.js (Main Entry File)

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
import Layout from "./pages/Layout";
import Home from "./pages/Home";
import Blogs from "./pages/Blogs";
import Contact from "./pages/Contact";
import NoPage from "./pages/NoPage";

export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Home />} />
          <Route path="blogs" element={<Blogs />} />
          <Route path="contact" element={<Contact />} />
          <Route path="*" element={<NoPage />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

4. pages/Layout.js

```
import { Outlet, Link } from "react-router-dom";
const Layout = () => {
  return (
<>
<nav>
<ul>
```

```
<li><Link to="/">Home</Link></li>

<li><Link to="/blogs">Blogs</Link></li>
<li><Link to="/contact">Contact</Link></li>
</ul>
</nav>
<Outlet />
</>
  );
};

export default Layout;
```

5. pages/Home.js

```
const Home = () => {
  return <h1>This is Home Page</h1>;
};
export default Home;
```

6. pages/Blogs.js

```
const Blogs = () => {
  return <h1>Blog Articles</h1>;
};
export default Blogs;
```

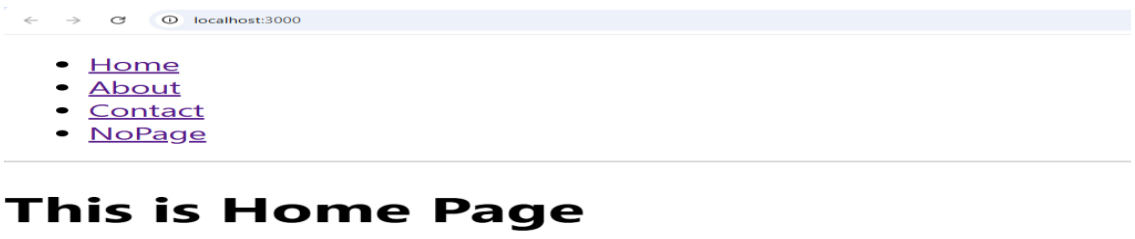
7. pages/Contact.js

```
const Contact = () => {
  return <h1>Contact Me</h1>;
};
export default Contact;
```

8. pages/NoPage.js

```
const NoPage = () => {
  return <h1>404 - Page Not Found</h1>;
};
export default NoPage;
```

Output:



13.Build User Authentication and Authorization using Spring Security

Software Required

- Java JDK 17
- Spring Tool Suite (STS) / IntelliJ IDEA
- Maven
- Web Browser

Step 1: Create Spring Boot Project

Go to **Spring Initializr** and select:

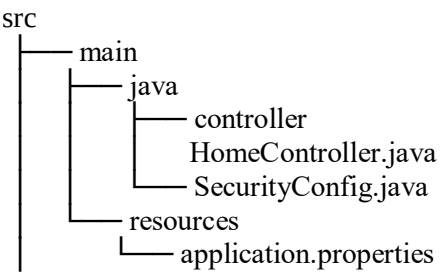
- Project: Maven
- Language: Java
- Spring Boot: Latest Stable Version

Add Dependencies

- Spring Web
- Spring Security

Click **Generate** and open the project in STS.

Step 2: Project Structure



Step 3: Security Configuration

Create **SecurityConfig.java**

packagecom.example.demo;

importorg.springframework.context.annotation.Bean;
importorg.springframework.context.annotation.Configuration;
importorg.springframework.security.config.Customizer;
importorg.springframework.security.config.annotation.web.builders.HttpSecurity;
importorg.springframework.security.core.userdetails.User;
importorg.springframework.security.core.userdetails.UserDetails;
importorg.springframework.security.provisioning.InMemoryUserDetailsManager;

```
importorg.springframework.security.web.SecurityFilterChain;

@Configuration

publicclassSecurityConfig {

    @Bean
publicInMemoryUserDetailsManageruserDetailsService() {

UserDetailsuser=User.withDefaultPasswordEncoder()
.username("student")
.password("1234")
.roles("USER")
.build();

UserDetailsadmin=User.withDefaultPasswordEncoder()
.username("admin")
.password("admin123")
.roles("ADMIN")
.build();

returnnewInMemoryUserDetailsManager(user, admin);
    }

    @Bean
publicSecurityFilterChainsecurityFilterChain(HttpSecurityhttp)
throwsException {

http
.authorizeHttpRequests(auth ->auth
.requestMatchers("/").permitAll()
.requestMatchers("/user").hasRole("USER")
.requestMatchers("/admin").hasRole("ADMIN")
.anyRequest().authenticated()
            )
.formLogin(Customizer.withDefaults());

returnhttp.build();
    }
}
```

Step 4: Create Controller

```
packagecom.example.demo.controller;

importorg.springframework.web.bind.annotation.GetMapping;
importorg.springframework.web.bind.annotation.RestController;

@RestController
publicclassHomeController {

    @GetMapping("/")
```

```
publicStringhome() {
return"Welcome Everyone";
    }

    @GetMapping("/user")
publicStringuser() {


return"Welcome Student";
    }

    @GetMapping("/admin")
publicStringadmin() {
return"Welcome Admin";
    }
}
```

Step 5: Run the Project

Run the application as:

Spring Boot App

Step 6: Test in Browser

Home Page

http://localhost:8080/

Output

Welcome Everyone

User Page

http://localhost:8080/user

Login

Username : student
Password : 1234

Output

Welcome Student

Admin Page

http://localhost:8080/admin

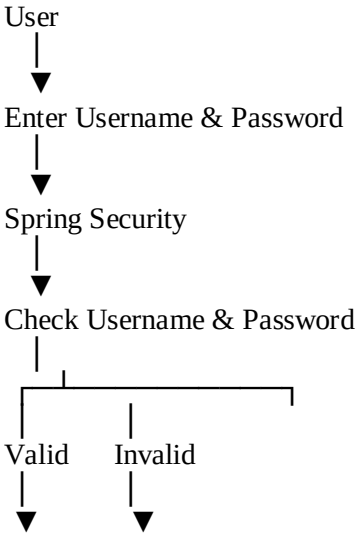
Login

Username : admin
Password : admin123

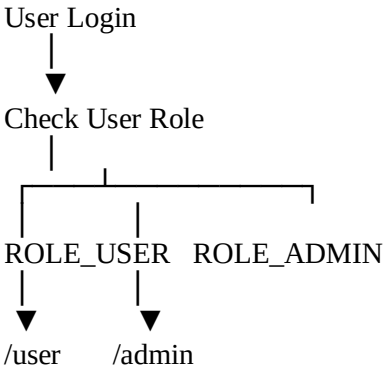
OUTPUT

Welcome Admin

Authentication Flow



Authorization Flow



Working

- **Authentication** verifies the user's identity by checking the username and password.
- **Authorization** checks the user's role (USER or ADMIN) before allowing access to protected pages.
- The **student** user can access only /user.
- The **admin** user can access only /admin.
- The home page (/) is accessible to everyone.

Expected Output

URL	Username	Password	Result
/	No Login	No Login	Welcome Everyone
/user	student	1234	Welcome Student
/admin	admin	admin123	Welcome Admin

14.Write a Spring Boot application with a REST controller that add two numbers.(in Spring tool suite)

- **STEPS**

1.open vs code
2.press ctrl+shift+p
3. Search Spring Initializr: Create a Maven Project.
4.Select:
.Language: Java
.Spring Boot: latest stable version
.Group: com.example
.Artifact: HelloSpringBoot
.Packaging: Jar
.Java: 17 or newer
5.Select dependency:
Spring Web
6.Choose a folder and open the generated project in VS Code.

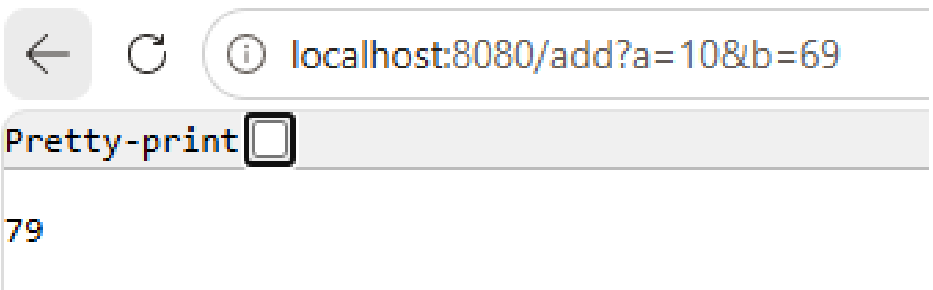
7. Create the CalcController.java
Inside:
src/main/java/com/example/demo/

```
package com.example.demo;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
@RestController
public class CalcController {

   @GetMapping("/add")
   public int add(@RequestParam int a, @RequestParam int b) {
      return a + b;
   }
}
```

8.Run the code : ./mvnw spring-boot:run

Output:



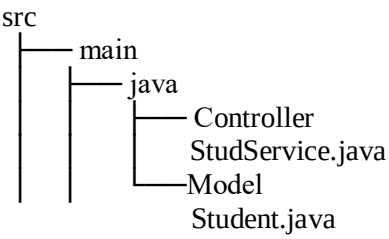
15.Create Student.java – an entity class with student information. Design a Rest Controller with GET, PUT, POST and DELETE Restful APIs.

//DemoTest2Application.java

1.open vs code
2.press ctrl+shift+p
3. Search Spring Initializr: Create a Maven Project.
4.Select:
.Language: Java
.Spring Boot: latest stable version
.Group: com.example
.Artifact: HelloSpringBoot
.Packaging: Jar
.Java: 17 or newer

5.Select dependency:
Spring Web
6.Choose a folder and open the generated project in VS Code.

7. Create the Controller folder and Model
Like this



Student.java

```
package com.example.demotest2.Model;


public class Student {
        private String name;
        private String USN;
        private String dept;
        private String  sem;
        public void setName(String name) {
        this.name = name;
        }

        public void setUSN(String USN) {
        this.USN = USN;
        }

        public void setDept(String dept) {
```

```
        this.dept = dept;
        }

        public void setSem(String sem) {
        this.sem = sem;
        }


        public Student() {
        }

        public String getName() {
        return name;
        }

        public String getUSN() {
        return USN;
        }

        public String getDept() {
        return dept;
        }

        public String getSem() {
        return sem;
        }
public Student(String name, String USN, String dept, String sem) {
        this.name = name;
        this.USN = USN;
        this.dept = dept;
        this.sem = sem;
        }
}
```

Student.Java
StudService.java

```
package com.example.demotest2.Controller;
import com.example.demotest2.Model.Student;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/student")
public class StudService {
        Student s1;
   @GetMapping("{name}")
        public Student getStudentDetails(String name) {
        return s1;
        //new Student("alia", "12", "cs", "5");
        }
        @PostMapping
```

```
        public String  createStudentDetails(@RequestBody Student s1) {
        this.s1 = s1;
        return "created successfully";
        }
        @PutMapping
public String updateStudentDetails(@RequestBody Student s1){
        this.s1=s1;
        return "updated successfully";

}
@DeleteMapping("{name}")
public String deleteStudentDetails(String name)
{
        this.s1=null;
        return "deleted successfully";
}
}
```

8.Run the code
./mvnw spring-boot:run

9.In postman Body-raw-JSON and write

```
{
"name":"ram",
"USN":"109",
"dept":"CSE",
"sem":"7"
}
```

OUTPUT:

GET
http://localhost:8082/student

```
{
   "timestamp": "2026-09-10T04:38:22.606Z",
   "status": 405,
   "error": "Method Not Allowed",
   "path": "/student"
}
```

PUT
http://localhost:8082/student

updated successfully

POST
http://localhost:8082/student

created successfully

16.Write a java program to connect to a mysql database, retrieve data from a table and display it in the console. (install Mysql connectot/jar download)

1.create a folder (javaMySqlProject)
2.inside folder create a file student.java write code
3.go to Referenced libraries then paste Mysql connector path
4.lastly create a database & Run without debugging

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.Statement;

	public class mySql{
	   public static void main(String[] args) throws Exception {
	      // 1. Register and load the Driver
Class.forName("com.mysql.cj.jdbc.Driver");

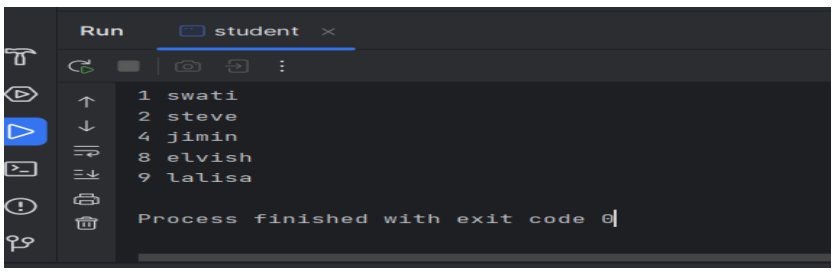
	      // 2. Build the connection with DriverManager.getConnection
	      Connection connection = DriverManager.getConnection(
	            "jdbc:mysql://localhost:3306/collage",
	            "username",
	            "password");

	      // 3. Create a statement
	      Statement statement = connection.createStatement();

	      // 4. Execute the query and fetch the result set
// 4. Execute the query and fetch the result set
ResultSetresultSet = statement.executeQuery("select * from student");

	      // 5. Traverse through the resultset
while(resultSet.next()){
System.out.println(resultSet.getInt(1)+" "
	                  + resultSet.getString(2)+" ");
	      }
	      // 6. Close the connection
connection.close();
}
}
```

Output:



17. Create a Java application to insert new records into an existing.

```
import java.sql.*;
import java.sql.Connection;
import java.sql.DriverManager;

public class sql {

public static void main(String[] args) throws Exception {

        String url="jdbc:mysql://localhost:3306/company";
        String uname="username";
        String pass="password";
      String name="ele";
      int id=4;
      String query="INSERT INTO employee (id,name) VALUES (?,?)";
      // 1. Register and load the Driver
      Class.forName("com.mysql.cj.jdbc.Driver");

      // 2. Build the connection with DriverManager.getConnection Connection connection =
      DriverManager.getConnection(url, uname, pass);

      // 3. Create a statement
      PreparedStatementst= connection.prepareStatement(query);
      st.setInt(1, id);
      st.setString(2, name);

      // 4. Execute the query and fetch the result set

      intval = st.executeUpdate();
      System.out.println(val+"rows affected");

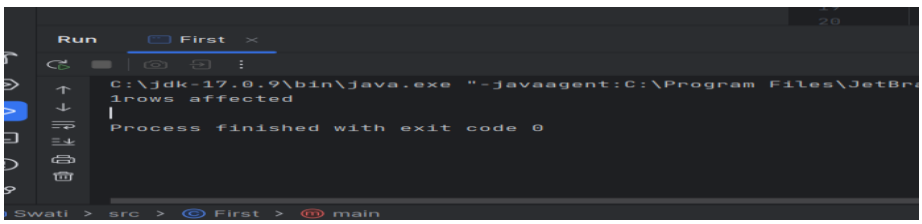
      // 5. Traverse through the resultset

      // 6. Close the connection

      connection.close();

      }
}
```

OUTPUT:



18. Write a JDBC program that uses a prepared statement to update.

//records in a database based on user input.

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;

public class pre {

        public static void main(String[] args) {

            // Database connection parameters

            String url = "jdbc:mysql://localhost:3306/collage";

            String username = "root";

            String password = "";

            try {

             // Connect to the database

             Connection connection = DriverManager.getConnection(url, username, password);



             // Prepare a SQL update statement with a placeholder

             String updateQuery = "UPDATE student SET name = ? WHERE id = ?";
PreparedStatement preparedStatement = connection.prepareStatement(updateQuery);

             // Get user input for the values to update

             Scanner scanner = new Scanner(System.in);

             System.out.print("Enter ID of the record to update: ");

             int id = scanner.nextInt();

             scanner.nextLine(); // Consume the newline character

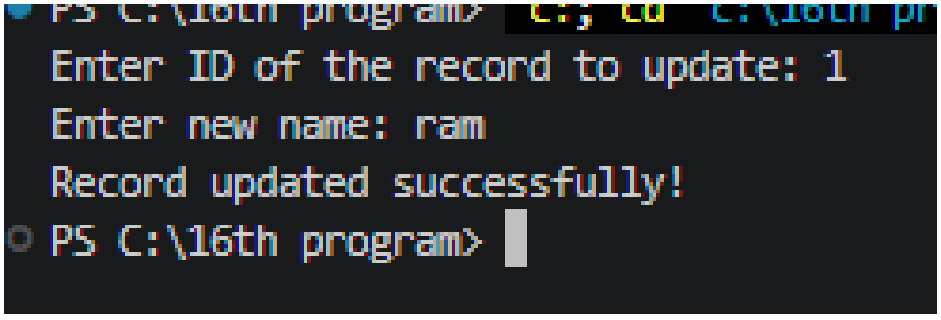
             System.out.print("Enter new name: ");

             String name = scanner.nextLine();
```

```
            // Set the values in the prepared statement
            preparedStatement.setString(1, name);
            preparedStatement.setInt(2, id);
        // Execute the update
            int rowsUpdated = preparedStatement.executeUpdate();

            if (rowsUpdated !=0) {
System.out.println("Record updated successfully!");
            } else {
            System.out.println("No records updated.");
            }
// Close resources
            preparedStatement.close();
            connection.close();
           } catch (SQLException e) {
            e.printStackTrace();
           }
       }
}
```

OUTPUT:



19. Perform the following operations using MongoDB

i. Create an employee database.
ii. Insert 2 records into the employee collection with suitable fields.
iii. Sort the employee by name
iv. Search the employee by Id
v. Remove the employee by Id

Step 1 - Open Command prompt
Step 2 - Start MongoDB Shell
Type: mongosh
Step 3 - Create Employee database

Type:
use employee

OUTPUT:
switched to db employee

Step 4 - Insert 2 records into Employee collection
Type:

```
db.Employee.insertMany([
  {id: 41, name: "John Deo", age: 18, department: "CS"},
  {id: 51, name: "Jane Smith", age: 20, department: "IT"}
])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...')
  }
}
```

Step 5 - Display the employee records

Type:

```
db.Employee.find()
```

OUTPUT:

```
[
  {
    id: 41,
    name: 'John Deo',
    age: 18,
    department: 'CS'
  },
  {
    id: 51,
    name: 'Jane Smith',
    age: 20,
    department: 'IT'
  }
```

```
]
```
Step 6 - Sort employees by name
Type:
```
db.Employee.find().sort({name: 1})
```

OUTPUT:
```
[
  {
    id: 51,
    name: 'Jane Smith',
    age: 20,
    department: 'IT'
  },
  {
 id: 41,
    name: 'John Deo',
    age: 18,
    department: 'CS'
  }
]
```

Step 7 - Search employee by ID
Type:
```
db.Employee.find({id: 41})
```

OUTPUT:
```
[
  {
    id: 41,
    name: 'John Deo',
    age: 18,
    department: 'CS'
  }
]
```
Step 8 - Remove employee by ID

Type:
```
db.Employee.deleteOne({id: 51})
```

OUTPUT:
```
{
  acknowledged: true,
  deletedCount: 1
}
```

Step 9 - Display remaining employee
Type:
```
db.Employee.find()
```

OUTPUT:
```
[
  {
    id: 41,
```

```
    name: 'John Deo',
    age: 18,
    department: 'CS'
  }
]
```

20. Perform the following operations using MongoDB Create Library_management database.

Step 1 - Open Command Prompt

Step 2 - Start MongoDB Shell

Type:
mongosh

You should get:
test>

Step 3 - Create Library Management database

Type:
use library_management

Output:
switched to db library_management

Step 4 - Create Students collection

Type:
db.createCollection("students")

Output:
{ ok: 1 }

Step 5 - Insert student records

Type:

```
db.students.insertMany([
  {regNo: "STU001", name: "Arun Kumar", dept: "CSE", year: 3},
  {regNo: "STU002", name: "Meera Devi", dept: "ECE", year: 2}
])
```

OUTPUT:

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...')
  }
```

Step 6 - Create Books collection

Type:
db.createCollection("books")

OUTPUT:

{ ok: 1 }

Step 7 - Insert book records

Type:
```
db.books.insertMany([
  {
    isbn: "9780132350884",
    title: "Clean Code",
    author: "Robert C. Martin",
    copies: 5
  },
  {
    isbn: "9780134494166",
    title: "Clean Architecture",
    author: "Robert C. Martin",
    copies: 3
  }
])
```

OUTPUT:
```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('...'),
    '1': ObjectId('...')
  }
}
```

Step 8 - Display all students

Type:
db.students.find().pretty()

OUTPUT:
```
[
  {
    regNo: 'STU001',
    name: 'Arun Kumar',
    dept: 'CSE',
    year: 3
  },
  {
    regNo: 'STU002',
    name: 'Meera Devi',
    dept: 'ECE',
    year: 2
  }
```

```
]
```

Step 9 - Display all books

Type:
```
db.books.find().pretty()
```

OUTPUT:
```
[
  {
    isbn: '9780132350884',
    title: 'Clean Code',
    author: 'Robert C. Martin',
    copies: 5
  },
  {
    isbn: '9780134494166',
    title: 'Clean Architecture',
    author: 'Robert C. Martin',
    copies: 3
  }
]
```

Step 10 - Update student year
Change STU001 year from 3 to 4.

Type:
```
db.students.updateOne(
  {regNo: "STU001"},
  {$set: {year: 4}}
)
```

OUTPUT:
```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
}
```

Step 11 - Update book copies
Change Clean Code copies from 5 to 7.

Type:
```
db.books.updateOne(
  {isbn: "9780132350884"},
  {$set: {copies: 7}}
)
```

OUTPUT:
```
{
  acknowledged: true,
  matchedCount: 1,
  modifiedCount: 1
```

```
}
```

Step 12 - Delete a student
Delete student STU002.
Type:
db.students.deleteOne({regNo: "STU002"})

OUTPUT:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

Step 13 - Delete a book
Delete the book with ISBN 9780134494166.
Type:
db.books.deleteOne({isbn: "9780134494166"})

Output:

```
{
  acknowledged: true,
  deletedCount: 1
}
```

Step 14 - Display final student records

Type:
db.students.find().pretty()

OUTPUT:

```
[
  {
    regNo: 'STU001',
    name: 'Arun Kumar',
    dept: 'CSE',
    year: 4
  }
]
```

Step 15 - Display final book records

Type:
db.books.find().pretty()

OUTPUT:

```
[
  {
    isbn: '9780132350884',
    title: 'Clean Code',
    author: 'Robert C. Martin',
    copies: 7
  }
]
```